PROGRAM-2
HEALTH MONITORING NODE

SOURCE CODE:

```
#include <Wire.h>

#define LM75_ADDRESS 0x48
#define PULSE_PIN 8

unsigned long pulseCount = 0;
unsigned long lastTempTime = 0;
unsigned long lastBpmTime = 0;

int previousPulseState = LOW;

float readTemperature()
{
   Wire.beginTransmission(LM75_ADDRESS);
   Wire.write(0x00);
   Wire.endTransmission();

   Wire.requestFrom(LM75_ADDRESS, 2);

   if (Wire.available() >= 2)
   {
      byte msb = Wire.read();
      byte lsb = Wire.read();

      int16_t rawTemperature =
         ((int16_t)msb << 8) | lsb;
```

```
        return rawTemperature / 256.0;
    }

    return -999.0;
}

void setup()
{
    Serial.begin(9600);
    Wire.begin();

    pinMode(PULSE_PIN, INPUT);

    Serial.println("I2C HEALTH MONITORING NODE");
    Serial.println("---------- ");
}

void loop()
{
    unsigned long currentTime = millis();

    /* Pulse counting */
    int currentPulseState = digitalRead(PULSE_PIN);

    if (currentPulseState == HIGH &&
        previousPulseState == LOW)
    {
        pulseCount++;
    }
```

```
previousPulseState = currentPulseState;

/* Temperature acquisition every 1 second */
if (currentTime - lastTempTime >= 1000)
{
   lastTempTime = currentTime;

   float temperature = readTemperature();

   Serial.print("Temperature: ");

   if (temperature > -100)
   {
      Serial.print(temperature, 1);
      Serial.println(" C");
   }
   else
   {
      Serial.println("Sensor Error");
   }
}

/* BPM calculation every 5 seconds */
if (currentTime - lastBpmTime >= 5000)
{
   lastBpmTime = currentTime;

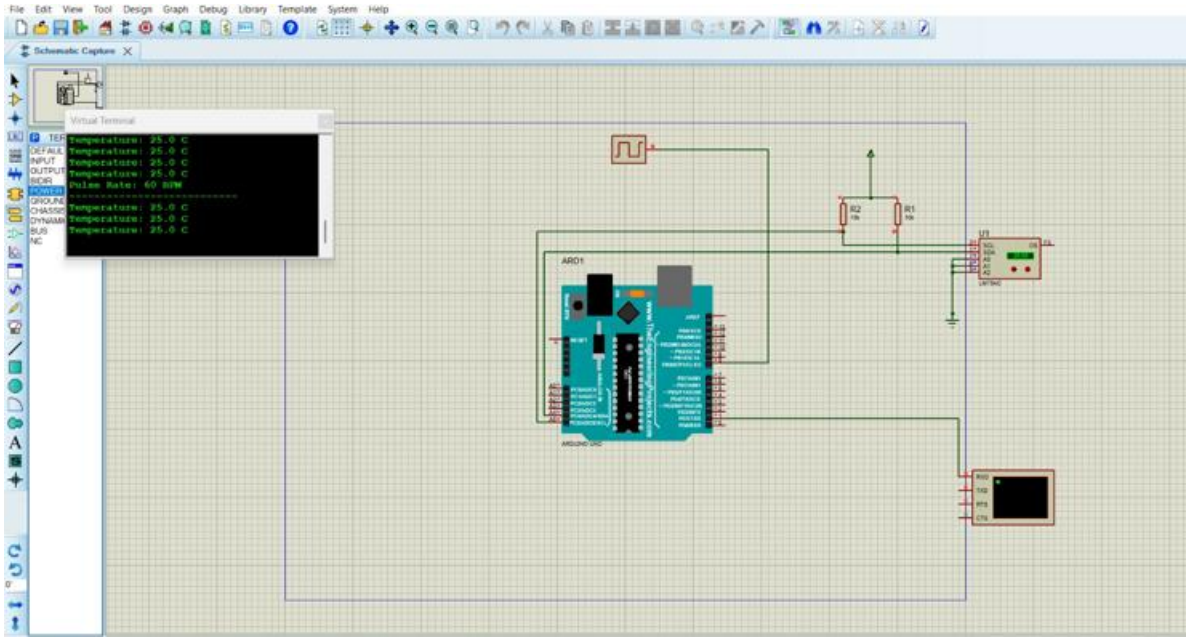
   float bpm = (pulseCount * 60.0) / 5.0;

   Serial.print("Pulse Rate: ");
```

```
        Serial.print(bpm, 0);
        Serial.println(" BPM");

        Serial.println("------------------------ ");

        pulseCount = 0;
    }
}
```



OBSERVATION:

- The LM75 temperature sensor continuously measures temperature and sends the collected data to the microcontroller through the I²C communication protocol.
- The system reads temperature values at regular one-second intervals to provide continuous temperature monitoring.
- The pulse sensor detects heartbeat signals by identifying changes in the pulse waveform and helps monitor the user's heart rate.
- The microcontroller calculates the heart rate in beats per minute (BPM) by processing the detected pulse signals at five-second intervals.
- The `millis()` function is used to schedule temperature measurements and pulse monitoring without interrupting the execution of other program operations.

- The measured temperature and calculated heart rate are displayed on the Serial Monitor, allowing users to observe the health-related sensor readings in real time.
- The program checks the availability of temperature data and displays an appropriate error message whenever the sensor fails to provide valid readings.
- The microcontroller processes the information received from the temperature and pulse sensors to support continuous monitoring of basic health parameters.
- The system helps observe variations in temperature and heart rate, providing useful information for basic health tracking and identifying readings that may require further attention.
- The developed health monitoring node demonstrates the integration of temperature and pulse sensors with a microcontroller and can serve as a foundation for advanced healthcare monitoring and remote patient monitoring applications.